



OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

OBJECT ORIENTED SOFTWARE ENGINEERING

Pattern Categories

Archana A

Department of Computer Applications

we group patterns into **three** categories:

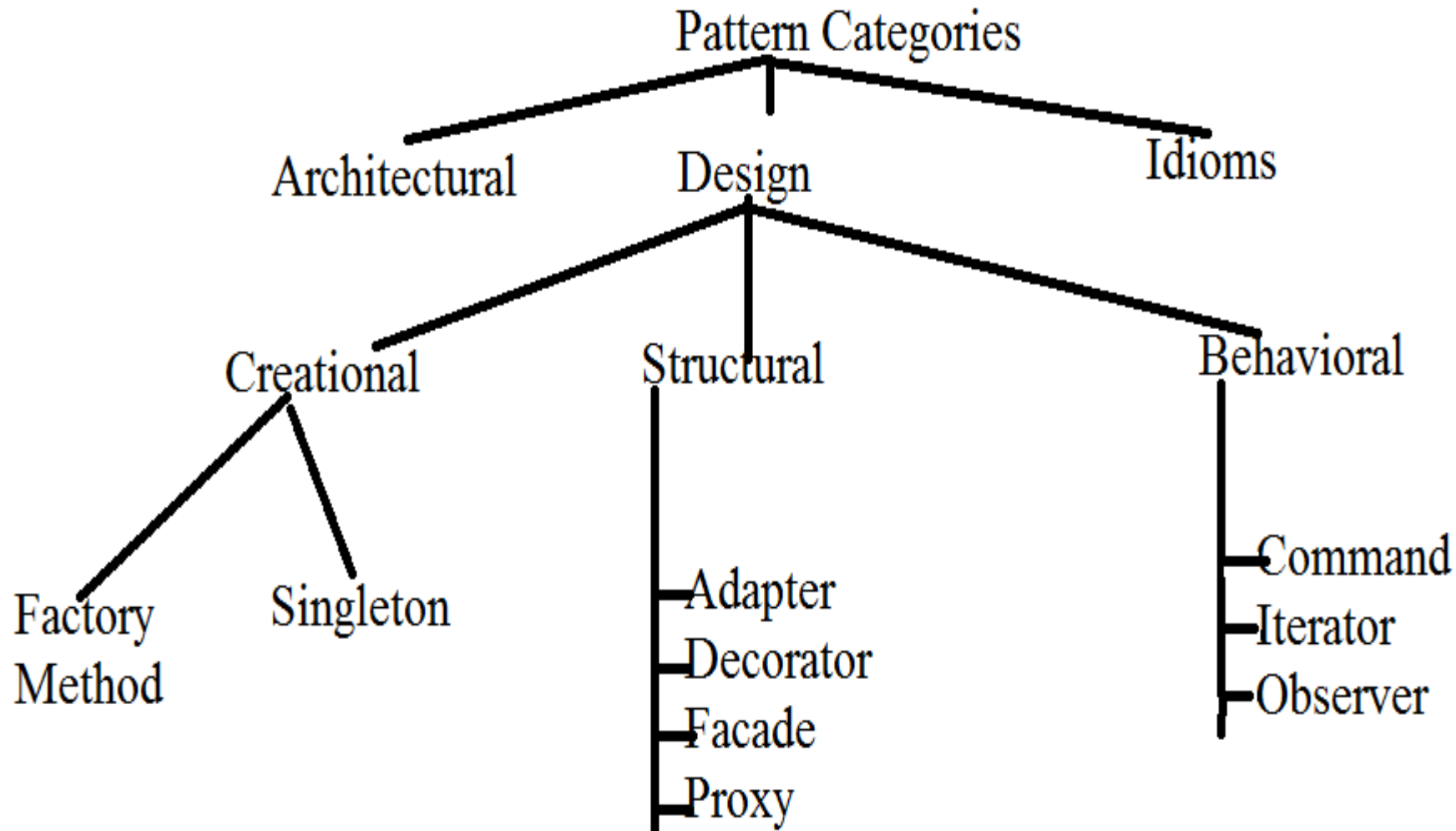
- **Architectural patterns**
- **Design patterns**
- **Idioms**

- Each category consists of patterns having a similar range of scale or abstraction.

As per the design pattern reference book **Design Patterns - Elements of Reusable Object-Oriented Software** , there are **23 design patterns**.

These 23 patterns can be classified in **three categories**:

1. **Creational**
2. **Structural**
3. **Behavioral patterns**



- Architectural patterns are used to describe **viable software architectures** that are built according to some overall structuring principle.

Definition:

- An architectural pattern expresses **a fundamental structural organization schema for software systems**.
- It provides a set of **predefined subsystems, specifies their responsibilities, and includes rules and guidelines** for organizing the relationships between them.
- Example: **Model-view-controller pattern (MVC)**

- Design patterns are used to describe **subsystems of a software architecture as well as the relationships between them** (which usually consists of several smaller architectural units)

Definition:

- A design pattern provides a scheme for refining the subsystems or components of a software system, or the relationships between them.

- It describes a **commonly-recurring structure of communicating components** that solves a general design problem within a particular Context.
- They are **medium-scale patterns**. They are smaller in scale than architectural patterns, but tend to be independent of a particular programming language or programming paradigm.
- Example: **Publisher-Subscriber pattern**

- Idioms deals with the **implementation of particular design issues**.

Definition:

- An idiom is a **low-level pattern** specific to a programming language.
- An idiom describes how to implement particular aspects of components or the relationships between them using the features of the given language.
- Idioms represent the **lowest- level patterns**. They address aspects of both design and implementation.
- Eg: **Counted body pattern**

OBJECT ORIENTED SOFTWARE ENGINEERING

Pattern Description

Archana A

Department of Computer Applications

1. **Name** : The **name** and a short summary of the pattern.
2. **Also known as**: **Other names** for the pattern, if any are known.
3. **Example** : A real world example demonstrating the **existence of the problem** and the need for the pattern.
4. **Context** :The **situations** in which the patterns may apply.
5. **Problem** : The problem the pattern addresses, including a discussion of its associated forces.

6. Solution : The fundamental solution principle underlying the pattern.

7. Structure : A detailed specification of the structural aspects of the pattern, including **CRC** – cards for each participating component and an **OMT class diagram**.

8. Dynamics : Typical scenarios describing the run time behavior of the pattern.

9. Examples resolved: Discussion for any important aspects for resolving the example that are not yet covered in the solution , structure, dynamics and implementation sections.

-
- 10. Variants:** A brief description of variants or **specialization** of a pattern
 - 11. Known uses:** Examples of the use of the pattern, taken from existing systems.
 - 12. Consequences:** The benefits the pattern provides, and any potential liabilities.
 - 13. See Also:** References to patterns that solve similar problems, and the patterns that help us refine the pattern we are describing



THANK YOU

Ms. Archana A

Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392